

Telemetry Model

A Nasdanika Zephyr Book

Pavel Vlasov

Version 2026.11.0, 2026-11-07

Table of Contents

Colophon . . . 1

Preface . . . 2

Who this book is for . . . 2

A book that stands on its own . . . 2

What you need . . . 3

Conventions . . . 3

I: The Why . . . 4

1. Every signal is precious . . . 5

1.1. What a model can say that a log line cannot . . . 5

1.2. Operational and strategic . . . 5

1.3. Choosing what to keep . . . 5

1.4. What it is for . . . 6

2. Faithful to OpenTelemetry, navigable as a graph . . . 7

2.1. Why a model and not a format . . . 7

3. The landscape . . . 8

II: The How . . . 9

4. Getting started . . . 10

4.1. Two modules, and why . . . 10

4.2. Adding the exporters . . . 10

4.3. What the build does . . . 12

5. The model . . . 13

5.1. The shape of OTLP . . . 13

5.2. Span . . . 14

5.3. Changes . . . 14

5.4. Logs and metrics . . . 14

6. Exporting from the SDK . . . 16

6.1. An SDK that exports to models . . . 16

6.2. To a resource, or to a consumer . . . 17

6.3. What the exporters do on the way . . . 18

7. Recording changes . . . 19

7.1. The model that changes . . . 19

7.2. A unit of work . . . 19

7.3. What is stored . . . 20

7.4. When nothing is recorded . . . 20

8. Reading telemetry back . . . **22**
9. Recording a unit of work . . . **24**
9.1. Where the runtime records . . . 25
III: Looking Ahead . . . **26**
10. Looking ahead . . . **27**
10.1. Queries . . . 27
10.2. Replay . . . 27
10.3. Attributes that resolve . . . 27
10.4. Live views . . . 27
10.5. Higher floors . . . 27
Appendix A: Reference . . . **28**
A.1. Packages . . . 28
A.2. Java classes . . . 28
A.3. Formats . . . 29
Appendix B: Building this book . . . **30**
Appendix C: Glossary . . . **31**

Colophon

Telemetry Model: A Nasdanika Zephyr Book

Edition 2026.11.0, built on 2026-11-07.

This edition describes the telemetry model of the Nasdanika SDK Runtime 2026.11.0, Maven coordinates `org.nasdanika.sdk.runtime.models.telemetry:model:2026.11.0`, namespace URI `runtime.sdk.nasdanika.org/models/telemetry` and the URIs of its `traces`, `logs` and `metrics` packages under it, and its exporters, `org.nasdanika.sdk.runtime.models.telemetry:exporters:2026.11.0`. Every listing in it is included from the exporters' tests, which are compiled and run when the book is built.

Copyright © 2026 Pavel Vlasov.

The text of this book is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). You may share and adapt it for any purpose, commercial purposes included, provided you give appropriate credit, provide a link to the license, and indicate if you made changes. A suggested credit: "Pavel Vlasov, *Telemetry Model: A Nasdanika Zephyr Book*, licensed under CC BY 4.0", with a link to github.com/Nasdanika-SDK/runtime.

The code is licensed under the Eclipse Public License 2.0, the license of the runtime. In addition, you may use the code of this book's listings in your own programs and documentation without asking for permission and without attribution.

Neither license covers the Nasdanika name or logo: CC BY 4.0 grants no trademark rights.

OpenTelemetry is a trademark of The Linux Foundation. Eclipse and EMF are trademarks of the Eclipse Foundation. Java is a trademark of Oracle and its affiliates. Other names may be trademarks of their owners. Their use here identifies the technologies the model relates to and implies no endorsement.

The source of this book and its tests are at github.com/Nasdanika-SDK/runtime, in `models/telemetry/exporters`; the model is in `models/telemetry/model`.

Typeset with Asciidoctor PDF in Noto Sans and M+ 1mn, in the look of the Nasdanika Zephyr Books.

Preface

This book is about OpenTelemetry signals as EMF models: traces, log records and metrics, kept next to the models they describe, in the same formats and the same repositories, instead of being shipped to a backend and forgotten on its schedule. It covers two modules of the Nasdanika SDK Runtime: the telemetry model, and the exporters that fill it from the OpenTelemetry SDK.

The first part of the book is the why: why some telemetry is worth keeping forever, what turns a stream of records into a graph, and where the model stands among the tools of observability. The second part is the how, for Java developers: an SDK that exports to models, a unit of work that records its changes to a model in its spans, the trace read back, and the same in one object. The third says what comes next.

Who this book is for

Developers and architects who need to explain, months or years later, what a command, a release, a migration or an agent did, and to what.

Builders of agents and tools who want the work of an agent to be reviewable and reversible: a trace whose spans are the agent's steps, with the changes each step made.

A book that stands on its own

This book is one of the Nasdanika Zephyr Books: in-depth manuals, one per module of the Nasdanika SDK Runtime, in the manner of IBM Redbooks. It covers the telemetry model and its exporters together, because they are used together: the exporters are a module of their own only to keep the OpenTelemetry SDK out of the model. You do not need any other book of the series, or the Nasdanika SDK Runtime book, to read this one: where the books touch, this one repeats what you need.

The series is named after the Bootswatch theme it is styled with, Zephyr, and has nothing to do with the Zephyr real-time operating system or with SmartBear's Zephyr test management.

What you need

- **A JDK for Java 25** and **Maven 3.9.16 or later**.
- The model's Java module is `org.nasdanika.sdk.runtime.models.telemetry`, with the Maven coordinates `org.nasdanika.sdk.runtime.models.telemetry:model:2026.11.0`. The exporters' module is `org.nasdanika.sdk.runtime.models.telemetry.exporters`, with the coordinates `org.nasdanika.sdk.runtime.models.telemetry:exporters:2026.11.0`.
- Some familiarity with OpenTelemetry: tracers, spans, loggers, meters, and the SDK that exports them.
- Some familiarity with EMF helps: an EMF model is a set of Java interfaces generated from a metamodel, with containment, references, reflection, persistence and change recording for free.

Conventions

- **Every listing is included from a test** of the exporters and compiled and run when the book is built. Callouts, such as <1>, explain the lines they mark.
- **Output is checked.** The trace printed in Chapter 8 is a file that the test compares with what the reader prints.
- **Status labels** mark what this edition does not have yet: PLANNED for designed but not built, SKETCH for partly built.
- **Voice.** I write in the first person.



> Editor's notes record inconsistencies between the model and its documentation found while writing. They are shown in builds for the author and left out of published editions.
>
> This book replaces the readmes of the model and the exporters, `models/telemetry/model/doc/readme.md` and `models/telemetry/exporters/doc/readme.md`, which are deleted. The model's POM was named "Nasdanika SDK Runtime File Model"; it is "Nasdanika SDK Runtime Telemetry Model" now.

Part I: The Why

Why some telemetry is worth keeping, why it should be a model, and where the model stands among observability tools. No code is needed to read this part.

Chapter 1. Every signal is precious

This model uses telemetry the opposite way from the mainstream. Mainstream observability collects enormous volumes cheaply, samples them, keeps them for days or weeks, and routinely drowns in its own signals. Here collection is deliberate and retention is indefinite: every signal is precious, and its price is paid knowingly, in exchange for being able to explain, years later, what happened.

1.1. What a model can say that a log line cannot

"Somebody changed the year of a book" is a log line. "In the course of the work named 'Correct the year', the attribute `year` of the book 'Domain-Driven Design' changed to 2004" is structure: the work, the object and the attribute are elements you can navigate to, and the record is stored next to the model, in Git.

Storage and compute are cheap at this scale. What is expensive is not being able to answer, two years later, who did what to what, and why it looks the way it does.

1.2. Operational and strategic

Operational telemetry answers "is it healthy right now". This model answers "who did what to what, and why". The two are complementary, and the same instrumented code feeds both: one exporter streams to an operational backend, sampled and forgotten; the other writes to the model, and it is kept.

1.3. Choosing what to keep

The discipline that keeps this affordable is choosing what is a unit of work worth keeping. A command that changes models, a release, an agent run, a migration: yes. Every request a server handles: no; that is operational telemetry, and it belongs in an operational backend, from the same instrumentation.

Counted, it is cheap. A command line run, an agent session or a build step produces hundreds to thousands of spans, which compress to kilobytes. Strategic telemetry is not the firehose; it is the record of work that mattered, and it costs less than the meeting held to reconstruct it.

1.4. What it is for

- **Audit and compliance.** Every change to an element during an audit period, who made it, and in the course of what work, is a query instead of an archaeology project.
- **Agents.** An agent's run is a trace whose spans are its tool calls, and the changes it made to models are in the spans that made them. Reviewing an agent's work means reading that trace, and reverting it means applying the changes in reverse.
- **Evidence.** Telemetry kept with the build or the release it describes is evidence of what ran. The runtime's command line launcher saves the telemetry of every launch as models, in a directory per launch.
- **Teaching.** Trace versus span versus metric is easier to show than to define. A small trace, saved as a model, can be browsed, diagrammed and walked through element by element.

Chapter 2. Faithful to OpenTelemetry, navigable as a graph

The model follows OpenTelemetry's own data model, the OpenTelemetry Protocol, OTLP, so that nothing is lost on the way in. Its classes are OTLP's messages, with OTLP's names and documentation: `TracesData`, `ResourceSpans`, `ScopeSpans`, `Span`, and so on for logs and metrics. Identifiers are lowercase hexadecimal, as the OTLP exporters write them, and times are nanoseconds since the UNIX epoch.

On top of OTLP, the model adds what makes telemetry a graph instead of a stream of records:

- **Spans nest.** A span contains its children within an instrumentation scope, so a trace reads as a tree. Across scopes, spans reference their parent by id.
- **Log records live in their spans.** A log record emitted in a span is contained by the span, not stored in a separate stream to be joined later.
- **Changes live in their spans.** A span contains the description of the model changes recorded while it was current.
- **References resolve.** Span links and the change classes reference spans as objects, not only by id, and every span knows what references it. An exemplar references the span that produced a measurement.

Spans are keyed by trace and span id, so their URIs are stable and readable.

2.1. Why a model and not a format

A trace saved as a model is something you load, navigate, query, compare and commit. It uses the formats every model of the runtime uses: YAML and JSON to read, XMI to interchange, EMF's binary format for size. And a change to a model, recorded in the span that made it, refers to the changed object by its URI, so it resolves to the model wherever the model is saved.

Chapter 3. The landscape

Observability platforms are built for the firehose: ingestion of high cardinality, sampling, short retention, questions about health now. This model is not a competitor and would be a poor one. It is for the small fraction of signals worth keeping, with the same instrumentation and a different exporter.

Observability tools for language models and agents keep traces of prompts and tool calls per session, which refer to names in someone else's service. Here a span lands next to the models the agent worked on, and its changes are in it.

OpenTelemetry itself says how signals travel; collectors and backends say where they go. Neither says what signals mean against a model of the system. This model is a destination that stays faithful to OpenTelemetry at its boundary.

Audit logs and W3C PROV. Audit logs have the instinct to retain, but no types and no joins. PROV has the right concepts, activity, agent and entity, as a separate vocabulary to adopt. Here provenance is part of the same models.

The status quo is grep over archives of logs, a wiki page titled "why did this change", and the person who remembers. The claim of this model is that the person who remembers can be a query.

Part II: The How

The model and its exporters in Java: adding them to a project, the four packages, exporting from the OpenTelemetry SDK, recording model changes in spans, reading telemetry back, and recording a unit of work in one object.

Chapter 4. Getting started

4.1. Two modules, and why

Instrumented code, the runtime's included, depends on the OpenTelemetry API only. The SDK is needed in one place: the assembly that runs a unit of work and decides where its telemetry goes. The exporters depend on the SDK, so they live in a module of their own, and the telemetry model and everything instrumented stay free of it.

Module	Who needs it
`org.nasdanika.sdk.runtime.models.telemetry`	Code that reads, writes or records into telemetry models; it depends on EMF and the OpenTelemetry API
`org.nasdanika.sdk.runtime.models.telemetry.exporters`	The assembly that runs a unit of work; it depends on the model, the runtime's `common` module and the OpenTelemetry SDK

4.2. Adding the exporters

The runtime's bill of materials, `org.nasdanika.sdk.runtime:bom`, manages the versions of the runtime's modules, of its models, and of their dependencies, OpenTelemetry among them. Import it in `dependencyManagement`, and depend on the exporters, which bring the model, and on the SDK, without versions:

```
<dependency>
    <groupId>org.nasdanika.sdk.runtime.models.telemetry</groupId>
    <artifactId>exporters</artifactId>
</dependency>
<dependency>
    <groupId>io.opentelemetry</groupId>
    <artifactId>opentelemetry-sdk</artifactId>
</dependency>
```

For telemetry in YAML or JSON, add the runtime's `org.nasdanika.sdk.runtime:emf-json`; XMI and EMF's binary format need nothing more.

The modules are named Java modules:

Listing 1. The module declaration of the telemetry model

```
module org.nasdanika.sdk.runtime.models.telemetry {

    exports org.nasdanika.sdk.runtime.models.telemetry;
    exports org.nasdanika.sdk.runtime.models.telemetry.impl;
    exports org.nasdanika.sdk.runtime.models.telemetry.util;

    exports org.nasdanika.sdk.runtime.models.telemetry.logs;
    exports org.nasdanika.sdk.runtime.models.telemetry.logs.impl;
    exports org.nasdanika.sdk.runtime.models.telemetry.logs.util;

    exports org.nasdanika.sdk.runtime.models.telemetry.metrics;
    exports org.nasdanika.sdk.runtime.models.telemetry.metrics.impl;
    exports org.nasdanika.sdk.runtime.models.telemetry.metrics.util;

    exports org.nasdanika.sdk.runtime.models.telemetry.traces;
    exports org.nasdanika.sdk.runtime.models.telemetry.traces.impl;
    exports org.nasdanika.sdk.runtime.models.telemetry.traces.util;

    requires transitive org.eclipse.emf.common;
    requires transitive org.eclipse.emf.ecore;
    requires transitive org.eclipse.emf.ecore.change;
    requires transitive io.opentelemetry.api;
    requires transitive io.opentelemetry.context;

    provides EPackage with
        TelemetryEPackageProvider,
        LogsEPackageProvider,
        MetricsEPackageProvider,
        TracesEPackageProvider;

}
```

The model provides its four packages as services, so a resource set created with the runtime's `NasdanikaResourceSet.createAndConfigure()` loads telemetry in any format the runtime reads.

Listing 2. The module declaration of the exporters

```
module org.nasdanika.sdk.runtime.models.telemetry.exporters {

    exports org.nasdanika.sdk.runtime.models.telemetry.exporters;

    requires transitive org.nasdanika.sdk.runtime.models.telemetry;
    requires org.nasdanika.sdk.runtime.common;
    requires transitive io.opentelemetry.api;
    requires transitive io.opentelemetry.context;
    requires transitive io.opentelemetry.sdk;
    requires transitive io.opentelemetry.sdk.common;
    requires transitive io.opentelemetry.sdk.trace;
    requires transitive io.opentelemetry.sdk.logs;
```

```
    requires transitive io.opentelemetry.sdk.metrics;

}
```

4.3. What the build does

The model is defined in Xcore, in `models/telemetry/model/src/main/resources/org/nasdanika/sdk/runtime/models/telemetry`: `telemetry.xcore`, `traces.xcore`, `logs.xcore` and `metrics.xcore`. Maven generates the Java sources into `src-gen` with the Xtext Maven plugin. The change recording classes extend those of EMF's change model, which the build reads from a copy in `src/main/xcore-deps`, because the standalone Xcore builder cannot resolve the original's relative references inside a jar. `Changes`, which records changes, is hand-written, in the `traces` package.

Chapter 5. The model

The model is four EMF packages, one per part of OTLP:

Package	Contents
`telemetry`	`Resource`, the entity that produced the signals; `InstrumentationScope`; and typed attribute values, `KeyValue` and `AnyValue`
`traces`	`TracesData`, `ResourceSpans`, `ScopeSpans`, `Span`, `SpanEvent`, `SpanLink`, `SpanStatus`, `SpanReference`, and the change recording classes `ChangeDescription`, `FeatureChange` and `ResourceChange`
`logs`	`LogsData`, `ResourceLogs`, `ScopeLogs`, `LogRecord`, `SeverityNumber`
`metrics`	`MetricsData`, `ResourceMetrics`, `ScopeMetrics`, the metrics `Gauge`, `Sum`, `Histogram`, `ExponentialHistogram` and `Summary`, their data points, and `Exemplar`

The namespace URIs are `runtime.sdk.nasdanika.org/models/telemetry`, `runtime.sdk.nasdanika.org/models/telemetry/traces`, `runtime.sdk.nasdanika.org/models/telemetry/logs` and `runtime.sdk.nasdanika.org/models/telemetry/metrics`.

5.1. The shape of OTLP

Every signal has the same three levels of grouping, as in OTLP:

- a data root, `TracesData`, `LogsData` or `MetricsData`;
- a group per resource, the entity that produced the signals, with its attributes, such as `service.name`;
- a group per instrumentation scope, the library that emitted them, with its name and version;

and then the spans, log records or metrics.

Attributes are `KeyValue` pairs whose values are `AnyValue`: a string, a boolean, an integer, a double, bytes, an array of values, or a list of key-value pairs, as in OTLP.

5.2. Span

A `Span` has OTLP's fields: `traceId`, `spanId`, `traceState`, `parentSpanId`, `name`, `kind`, `startTimeUnixNano` and `endTimeUnixNano`, `attributes`, `events`, `links`, `status` and `flags`, with the counts of what was dropped.

The model adds four features:

`children`

: The spans whose parent it is, within the same instrumentation scope, keyed by span id.

`logRecords`

: The log records emitted while it was current.

`changeDescription`

: The changes to models recorded while it was current (Chapter 7).

`referrers`

: The span references that point at it: links, exemplars, and change descriptions and their changes. The opposite of `SpanReference.span`.

`SpanReference` is a trace and span id with a resolved `span`. `SpanLink` is a span reference with attributes, as in OTLP.

5.3. Changes

`ChangeDescription`, `FeatureChange` and `ResourceChange` extend the classes of EMF's change model, and are span references: each knows the span in which it was recorded. A `ChangeDescription` also contains `removedObjects`, the objects removed in the recorded work, as they were when they were removed.

5.4. Logs and metrics

A `LogRecord` has OTLP's fields: times, `severityNumber` and `severityText`, the `body` as an `AnyValue`, `attributes`, an `eventName`, and the trace and span ids it was correlated with.

Metrics are `Gauge`, `Sum`, `Histogram`, `ExponentialHistogram` and `Summary`, with their data points, as in OTLP. An `Exemplar`, a sample measurement with its trace context, references the span that

produced it.

Chapter 6. Exporting from the SDK

The exporters are OpenTelemetry SDK exporters, plugged into the SDK like any other:

Class	Exports
`ModelSpanExporter`	Spans to `TracesData`. It is also the sink of recorded model changes
`ModelLogRecordExporter`	Log records to `LogsData`, or into the spans they were emitted in
`ModelMetricExporter`	Metrics to `MetricsData`
`TelemetryConverter`	The SDK's data to the model, field by field as the OTLP exporters convert it: ids as lowercase hexadecimal, times as nanoseconds since the UNIX epoch

6.1. An SDK that exports to models

Listing 3. An SDK with the model exporters (`Recorder.java`)

```
/**
 * The SDK owns the exporters, and the exporters own the telemetry resources: shutting
 * the SDK down exports what is pending and saves them. Log records emitted in a span go
 * into that span.
 */
static OpenTelemetrySdk openTelemetry(
        ModelSpanExporter spans,
        Resource logs,
        Resource metrics) {

    var service = io.opentelemetry.sdk.resources.Resource.create(
            Attributes.of(AttributeKey.stringKey("service.name"), "librarian"));
    var resource = io.opentelemetry.sdk.resources.Resource.getDefault().merge(service);

    var logRecords = new ModelLogRecordExporter(logs, spans);                ①
    var measurements = new ModelMetricExporter(metrics);
    return OpenTelemetrySdk.builder()
            .setPropagators(Telemetry.w3cPropagators())                      ②
            .setTracerProvider(SdkTracerProvider.builder()
                    .setResource(resource)
                    .addSpanProcessor(BatchSpanProcessor.builder(spans).build())
                    .build())
            .setLoggerProvider(SdkLoggerProvider.builder()
                    .setResource(resource)
                    .addLogRecordProcessor(BatchLogRecordProcessor.builder(logRecords).build())
                    .build())
            .setMeterProvider(SdkMeterProvider.builder()
                    .setResource(resource)
```

```
                    .registerMetricReader(PeriodicMetricReader.builder(measurements).build())
                    .build())
            .build();
}
```

① The log record exporter is given the span exporter, so that log records emitted in exported spans are added to those spans; log records emitted outside any span go to the logs resource.

② The W3C propagators let the unit of work continue a trace started in another process, and work it starts in another process continue it.

6.2. To a resource, or to a consumer

Each exporter works in one of two modes.

To an EMF resource. All exports go to one data root in the resource, grouped as in OTLP by resource and instrumentation scope; exporting to a loaded resource appends to it. The resource is saved on `flush()` and `shutdown()`. The save runs outside the current OpenTelemetry instance, so saving telemetry is not itself traced.

To a consumer. Each export is a new data root, handed to the consumer:

Listing 4. Exporting to a consumer (`TelemetryBookTests.java`)

```
List<TracesData> batches = new ArrayList<>();
ModelSpanExporter exporter = new ModelSpanExporter(batch -> {
    ModelSpanExporter.resolveParents(batch);                    ①
    batches.add(batch);
});
try (OpenTelemetrySdk openTelemetry = OpenTelemetrySdk.builder()
        .setTracerProvider(SdkTracerProvider.builder()
                .addSpanProcessor(BatchSpanProcessor.builder(exporter).build())
                .build())
        .build()) {
    Tracer tracer = openTelemetry.getTracer(Recorder.SCOPE);
    Telemetry.inSpan(tracer, "Shelve", null, span ->
        Telemetry.inSpan(tracer, "Label", null, step -> null));
}                                                               ②
```

① Spans exported in one batch are not nested by the exporter; `resolveParents` nests those whose parents are in the batch.

② Closing the SDK exports what is pending.

6.3. What the exporters do on the way

The span exporter nests spans in their parents and resolves span references before it saves. A parent usually ends after its children, so a child is exported first and moved into its parent when the parent arrives.

The log record exporter, given the span exporter, adds a log record emitted in a sampled span to that span, waiting for the span to be exported if necessary. Log records without a span go to the logs, and so do log records whose span was never exported, when the exporters shut down.

A metric exporter with a periodic reader accumulates a metric per collection: a time series of cumulative points, by default.

The SDK owns the exporters, and the exporters own the telemetry resources: closing the SDK exports what is pending and saves them.

Chapter 7. Recording changes

Change recording is EMF's, extended so that recorded changes know their spans. `Changes.record(target, body)` records the changes the body makes to a resource set, a resource or an object, and hands the description to the sink that is current in the OpenTelemetry context. The span exporter is such a sink: it puts each description into the span it was recorded in.

7.1. The model that changes

The unit of work of this chapter curates a library. The library is a small dynamic EMF package, built in the test so that the book needs no generated model of its own: a `Library` with a `name` and `books`, and a `Book` with a `name` and a `year`. Any EMF model records the same way.

7.2. A unit of work

Listing 5. A unit of work whose telemetry and changes are saved as models (`Recorder.java`)

```
static void curate(Path directory) throws IOException {
    NasdanikaResourceSet telemetry = NasdanikaResourceSet.createAndConfigure(); ①
    Resource traces = telemetry.createResource(uri(directory, "traces.xml"));
    Resource logs = telemetry.createResource(uri(directory, "logs.xml"));
    Resource metrics = telemetry.createResource(uri(directory, "metrics.xml"));
    ModelSpanExporter spans = new ModelSpanExporter(traces);

    try (OpenTelemetrySdk openTelemetry = openTelemetry(spans, logs, metrics);
            Scope telemetryScope = Telemetry.makeCurrent(openTelemetry);        ②
            Scope changesScope = Changes.makeCurrent(spans)) {                  ③

        Tracer tracer = openTelemetry.getTracer(SCOPE);
        Telemetry.inSpan(tracer, "Curate the shelf", null, span -> {
            NasdanikaResourceSet models = NasdanikaResourceSet.createAndConfigure();
            models.getPackageRegistry().put(Shelf.LIBRARY.getNsURI(), Shelf.LIBRARY);
            Resource shelf = models.createResource(uri(directory, "shelf.xml"));
            Changes.record(models, () -> {                                      ④
                shelf.getContents().add(Shelf.library());
                shelf.save(null);
                return null;
            });

            EObject library = shelf.getContents().getFirst();
            EObject ddd = Shelf.book(library, "Domain-Driven Design");
            return Telemetry.inSpan(tracer, "Correct the year", null, edit -> {
                return Changes.record(models, () -> {                           ⑤
                    ddd.eSet(Shelf.YEAR, 2004);
                    shelf.save(null);
```

```
                    return null;
                });
            });
        });
    }                                                                    ⑥
}
```

① The telemetry resources live in a resource set of their own, created before the instance is made current, so that saving them at shutdown is not itself traced.

② The SDK is the unit of work's instance. Everything below reports to it.

③ The span exporter becomes the sink of recorded changes for the unit of work.

④ Creating the library and saving it is recorded in the current span, "Curate the shelf".

⑤ Correcting the year is recorded in its own span, so the change lands there.

⑥ Closing the SDK exports what is pending and saves `traces.xml`, `logs.xml` and `metrics.xml`.

7.3. What is stored

EMF's change recorder produces a description that undoes the changes. The runtime stores one that describes them forward, detached from the model:

- Objects added in the body are copied into the description, as objects to attach.
- Objects removed in the body, and not added back, are moved into the description, so their state when they were removed is kept for audit.
- Other objects are referenced by proxies with their URIs, so the description can be saved in a resource of another resource set, and resolves to the model wherever the model is saved.

The description, its feature changes and its resource changes reference the spans that were current when they were created: the description the span current at the start of the recording, a feature change the span in which the feature of the object was first changed.

7.4. When nothing is recorded

Nothing is recorded without a current sink, or outside a sampled span. The same code runs unchanged in a unit of work that does not record, and costs nothing there.



`Recording` (Chapter 9) does not make its span exporter the sink of recorded changes, and does not expose it, so a unit of work that records changes builds

> the SDK by hand, as this chapter does. Consider making `Recording.makeCurrent()` also make `Changes` current, or adding a getter for the span exporter.
>
> Writing this book found a defect in the exporters: when the telemetry packages were first initialized on an export thread of the SDK, such as the batch span processor's, their initialization failed with a `NullPointerException`, because EMF's global package registry delegates by the thread's context class loader and those threads have none, and the classes stayed broken for the life of the JVM. `Recording` and the launcher did not see it, because they configure a resource set first. It is fixed: `AbstractModelExporter` initializes the packages when it is loaded. `TelemetryBookTests.consumer`, run on its own, is its test.

Chapter 8. Reading telemetry back

Recorded telemetry is a model, so reading it is ordinary EMF:

Listing 6. Reading traces and their changes (`TraceReader.java`)

```
static List<String> read(Path directory) {
    ResourceSet resourceSet = NasdanikaResourceSet.createAndConfigure();
    resourceSet.getPackageRegistry().put(Shelf.LIBRARY.getNsURI(), Shelf.LIBRARY); ①
    TracesData traces = (TracesData) resourceSet
            .getResource(Recorder.uri(directory, "traces.xml"), true)
            .getContents()
            .getFirst();

    // Spans nest within an instrumentation scope; across scopes they have parent ids
    Map<String, Span> spans = new LinkedHashMap<>();
    traces.getResourceSpans().stream()
        .flatMap(resourceSpans -> resourceSpans.getScopeSpans().stream())
        .flatMap(scopeSpans -> scopeSpans.getSpans().stream())
        .forEach(span -> collect(span, spans));
    Map<String, List<Span>> children = new LinkedHashMap<>();
    List<Span> roots = new ArrayList<>();
    for (Span span: spans.values()) {
        if (span.getParentSpanId() == null) {
            roots.add(span);
        } else {
            children.computeIfAbsent(span.getParentSpanId(), id -> new ArrayList<>()).add(span);
        }
    }

    List<String> lines = new ArrayList<>();
    roots.forEach(root -> print(root, children, "", lines));
    return lines;
}

static void print(
        Span span,
        Map<String, List<Span>> children,
        String indent,
        List<String> out) {

    out.add(indent + span.getName());
    ChangeDescription changes = span.getChangeDescription();                    ②
    if (changes != null) {
        for (EObject created: changes.getObjectsToAttach()) {
            out.add(indent + "  * created " + label(created));
        }
        changes.getObjectChanges().forEach(entry -> {
            for (FeatureChange change: entry.getValue()) {
                String feature = label(entry.getKey()) + "." + change.getFeatureName();
                out.add(indent + "  * " + feature + " = " + change.getValue());
            }
```

```
        });
    }
    children.getOrDefault(span.getSpanId(), List.of()).stream()
        .sorted(Comparator.comparingLong(Span::getStartTimeUnixNano))
        .forEach(child -> print(child, children, indent + "  ", out));
}
```

① The library's package, so that the copies of created objects in the change descriptions load as libraries and books, and the proxies of changed objects resolve to the saved model.

② The change description is a feature of the span. Its object changes are keyed by the changed objects.

For the unit of work of Chapter 7, it prints:

Listing 7. The trace of the unit of work

```
Curate the shelf
  * created Library 'A Modeler's Shelf'
  ResourceSetContributor.configure
  URIHandler.write
  Resource.save
  Correct the year
    * Book 'Domain-Driven Design'.year = 2004
    URIHandler.write
    Resource.save
```

The runtime's own spans, from another instrumentation scope, appear in the tree by their parent ids: the configuration of the models' resource set and the saves of the shelf happened inside "Curate the shelf" and "Correct the year", and the tree says so. The configuration of the telemetry resource set is not there, because it happened before the instance was made current.

Because the trace is a model, the same questions can be asked in a query language instead of a loop **PLANNED**.

Chapter 9. Recording a unit of work

`Recording`, in the exporters module, packages the SDK of Chapter 6 into one object: an OpenTelemetry SDK with the three model exporters, the log record exporter linked to the span exporter, and W3C propagators, recording to a traces, a logs and a metrics resource that are saved when it is closed.

Listing 8. Recording a unit of work (`TelemetryBookTests.java`)

```
Recording recording = Recording.builder()
        .serviceName("librarian")
        .directory(directory)                                          ①
        .build();
try (recording; Scope scope = recording.makeCurrent()) {               ②
    OpenTelemetry openTelemetry = Telemetry.current();
    Tracer tracer = Telemetry.tracer(openTelemetry);
    Telemetry.inSpan(tracer, "Import the catalog", null, span -> {
        Telemetry.log(
                Telemetry.logger(openTelemetry),
                Severity.INFO,
                null,
                "Two books found",
                null);                                                 ③
        return Telemetry.inSpan(tracer, "Index the books", null, step -> null);
    });
}                                                                      ④

TracesData traces = recording.getTraces();                             ⑤
Span work = traces.getResourceSpans().getFirst()
        .getScopeSpans().getFirst()
        .getSpans().getFirst();
assertThat(work.getName()).isEqualTo("Import the catalog");
assertThat(work.getChildren()).extracting(Span::getName).containsExactly("Index the books");
assertThat(work.getLogRecords())
    .extracting(log -> log.getBody().getStringValue())
    .containsExactly("Two books found");
```

① `traces.xml`, `logs.xml` and `metrics.xml` in the directory, which is created if it does not exist. `directory(path, extension)` chooses another format: `yml` or `json` with the runtime's `emf-json` module, `ebin` or `egz` for EMF's binary format. `resources(traces, logs, metrics)` takes resources made elsewhere.

② `makeCurrent()` makes the recording's instance the unit of work's: what `Telemetry.current()` returns.

③ A log record emitted in a span goes into the span.

④ Closing the recording shuts the SDK down, which exports what is pending and saves the

resources.

⑤ The model, complete once the recording is closed. `getLogs()` has the log records that were not in a recorded span, and `getMetrics()` the metrics.

`serviceName`, `serviceVersion` and `attribute` set the attributes of the resource the telemetry comes from.

9.1. Where the runtime records

The runtime's command line launcher records every command it runs with a `Recording`, in a directory per launch:

```
<home>/telemetry/2026/10/16-14-03-12-345/
  traces.yml
  logs.yml
  metrics.yml
```

The format is chosen with the system property `nsdk.telemetry.format` or the environment variable `NSDK_TELEMETRY_FORMAT`. Tests record with it what they, and the servers and browsers they start, did: the runtime's walk-throughs of generated web applications are recordings in which each step has its screenshots and the server's spans for the requests it caused.

Part III: Looking Ahead

Queries, replay, and telemetry that resolves to the models it names.

Chapter 10. Looking ahead

Everything in this chapter is **PLANNED**.

10.1. Queries

Questions about traces asked in a query language, not in Java loops: every change to an element during a period, the spans of an agent's run that changed a given model, the slowest steps of the releases of a quarter.

10.2. Replay

Spans carry timing and nesting, so an execution can be played back over a diagram of the system that ran it, with model changes appearing where they happened.

10.3. Attributes that resolve

Attributes of the semantic conventions, such as a code function or a URL, resolved to the model elements they name and recorded as markers. A span whose code attributes name a method resolves to that method in a model of the JVM, because both use the JVM's own names.

10.4. Live views

The telemetry of a running unit of work, watched live over server-sent events, and the same views over a recording after the fact.

10.5. Higher floors

Spans as work, resources as actors, metrics as entries of accounting: models of work, roles and accounting will connect to telemetry on their own floors, so that this model stays faithful to OpenTelemetry.

Appendix A: Reference

A.1. Packages

Package	Namespace URI
`org.nasdanika.sdk.runtime.models.telemetry`	`runtime.sdk.nasdanika.org/models/telemetry`
`org.nasdanika.sdk.runtime.models.telemetry.traces`	`runtime.sdk.nasdanika.org/models/telemetry/traces`
`org.nasdanika.sdk.runtime.models.telemetry.logs`	`runtime.sdk.nasdanika.org/models/telemetry/logs`
`org.nasdanika.sdk.runtime.models.telemetry.metrics`	`runtime.sdk.nasdanika.org/models/telemetry/metrics`

A.2. Java classes

Class	What it is for
`traces.Changes`	`record(target, body)` records the changes the body makes to a resource set, a resource or an object, and hands the description to the current sink. `makeCurrent(sink)` and `with(context, sink)` set the sink; `sink()` returns it.
`exporters.ModelSpanExporter`	Exports spans to `TracesData`, in a resource or to a consumer; nests spans and resolves span references; is a sink of recorded changes. `resolveParents(data)` nests the spans of a batch.
`exporters.ModelLogRecordExporter`	Exports log records to `LogsData`, or, given a span exporter, into the spans they were emitted in.
`exporters.ModelMetricExporter`	Exports metrics to `MetricsData`.
`exporters.TelemetryConverter`	Converts the SDK's data to the model.
`exporters.Recording`	An SDK with the three exporters, recording to three resources saved on `close()`.

A.3. Formats

Extension	Format
`yml`, `json`	YAML and JSON, with the runtime's `emf-json` module
`xml`, `xmi` and any other	XMI, the default of a resource set configured by the runtime
`ebin`, `egz`	EMF's binary format, and the same compressed

Appendix B: Building this book

This book is AsciiDoc in `models/telemetry/exporters/src/docs/asciidoc`, built with the exporters after their tests have passed:

```
mvn -pl models/telemetry/exporters -am verify                          # the exporters, their
tests and this book
mvn -pl models/telemetry/exporters verify -Dzephyr.editor-notes=false  # without editor's notes
mvn -pl models/telemetry/exporters verify -Dasciidoctor.skip=true      # without the book
```

The PDF and the single-page HTML are written to `target/book` and attached to the exporters' artifact with the classifier `book`; the test sources are attached with the classifier `test-sources`. The look comes from the runtime's `doc/zephyr` directory, shared by all the Nasdanika Zephyr Books.

Listings are tagged regions of the tests in `src/test/java/org/nasdanika/sdk/runtime/models/telemetry/exporters/book`, and of the module declaration. A listing line holds at most 96 characters, and a warning fails the build.

The trace in Chapter 8 is `curate.txt` in `src/test/resources/org/nasdanika/sdk/runtime/models/telemetry/exporters/book`, which `TelemetryBookTests` compares with what the reader prints. A change to the unit of work, or to the runtime's instrumentation, changes it; run the tests, review the difference, and update the file.

Appendix C: Glossary

Change description

EMF's record of changes to objects and resources. The runtime stores it forward and detached from the model, in the span it was recorded in.

Exemplar

A sample measurement of a metric, with the trace context it was taken in.

Exporter

The part of the OpenTelemetry SDK that sends finished telemetry somewhere: here, into models.

Instrumentation scope

The library or component that emitted telemetry, by name and version.

OTLP

The OpenTelemetry Protocol, and its data model, which the telemetry model follows.

Resource

In OpenTelemetry, the entity that produced telemetry, such as a service, described by attributes. Not an EMF resource, which is a file of a model.

Sink

The consumer of recorded change descriptions current in the OpenTelemetry context; the span exporter is one.

Span

A unit of work in a trace, with a name, a start and an end, attributes, events and links; here, also with its children, log records and recorded changes.

Unit of work

What a recording covers: a command, a release, an agent run, a migration.